

Assignment — Scale Out: From 2 GPUs to a Datacenter

Measure it · Design it · Investigate it · Explain it

Format: individual · **Duration:** one 3-hour session · **Weight:** one assignment grade **Where to run:** Kaggle Notebooks — free 2× T4 GPUs (Part 1; set the accelerator to GPU T4 ×2). Parts 2–5 are design / research / reasoning — no GPU needed. Optional bonus on RunPod.

The mission

You've learned the whole scaling ladder — DDP, FSDP, NCCL, tensor / pipeline / data parallelism, 3D parallelism, and running it on a cluster. This assignment proves you can **reason about training at any scale**:

- 1. **Measure it** on 2 GPUs — and explain every number.
- 2. **Design** the parallelism plan for a model that needs *thousands* of GPUs.
- 3. **Investigate** how a **real frontier model** was actually trained (the latest techniques).
- 4. **Explain** the concepts that tie it together.

The one line that defines a pass: "We can take any model + any cluster and say exactly how we'd split it, why, and what it would cost — and we can point to a real system that did it."

Coverage checklist (what this assignment makes you own)

DDP · FSDP / ZeRO · NCCL collectives · NVLink vs PCIe · Tensor Parallel · Pipeline Parallel + the bubble · Data Parallel · 3D parallelism · Expert Parallel (MoE) · Context Parallel (long ctx) · hardware mapping · Slurm / Kubernetes · rendezvous · DCGM · checkpointing & recovery

Part 1 · Measure it (hands-on — free 2× T4 on Kaggle) — 20 pts

You don't need your own GPUs — and there's nothing to set up. Upload `Kaggle_Multi_GPU_Labs.ipynb` to a Kaggle Notebook, set **Accelerator → GPU T4 ×2** (free, ~30 hrs/week), and hit **Run All**. It runs **all four labs** across both T4s and prints every number you need — no editing, no flags, no torchrun to type. (T4s use FP16 and the sizes are already tuned for 16 GB; the 1-GPU FSDP cell is meant to run out of memory — that's the lesson.)

Record each lab's result and write **2–3 sentences explaining what the number means**.

Lab	Run	Report + explain
DDP (<code>train_ddp.py</code>)	1 GPU, then 2	tokens/sec each · scaling efficiency = $\text{tps}(2)/(\text{2}\cdot\text{tps}(1))$ · <i>why is it below 1.0?</i>
FSDP (<code>train_fsdp.py</code>)	1 GPU (OOM), then 2	confirm the 1-GPU OOM · peak VRAM/GPU on 2 · <i>what three things got sharded?</i>
NCCL (<code>bench_nccl.py</code>)	2 GPUs	busbw at 256–512 MB · <i>how far below NVLink's ~900 GB/s, and why?</i>
TP from scratch (<code>tp_from_scratch.py</code>)	2 GPUs	the max error vs single-GPU · <i>why does column-parallel need no comm but row-parallel needs an all-reduce?</i>

Deliverable: a results table + your four explanations.

Part 2 · Design it (3D parallelism) — 20 pts

For **each** scenario below, produce a **parallelism plan**: choose the degrees, show the memory math, and **map each axis to the hardware** (what's inside a node on NVLink vs across nodes). Use the method from `EXERCISE_plan_the_parallelism.md` .

- 1. **Dense 34B · 32 GPUs** (4 nodes × 8 · 80 GB). → TP / PP / DP?
- 2. **MoE 600B, 8 experts active · 128 GPUs** (16 nodes × 8 · 80 GB). → now you also need **EP** — where does the expert all-to-all go?
- 3. **Dense 13B, but a 1M-token context · 16 GPUs** (2 nodes × 8 · 80 GB). → now you also need **CP** — why won't TP/PP alone solve this?

For each: **the plan** · **the memory math** · **the mapping** · **which collective each axis creates** (TP→all-reduce, EP→all-to-all, PP→point-to-point, DP→gradient all-reduce, CP→exchange K/V). More than one answer can be valid — **justify** it against memory + interconnect.

Part 3 · Investigate it (*the latest — reverse-engineer a real frontier model*) — 25 pts

Pick **one recent, openly-documented frontier model** and write a **1-page "parallelism teardown"** from its technical report / paper. This is where you meet the state of the art.

Pick **one** (or propose your own with a public report):

- **Kimi K3** (Moonshot) · **DeepSeek-V3 / R1** · **Llama 3 / 4** (Meta) · **Qwen 3** · **Mixtral / Mistral Large** · **GLM** · **Nemotron** (NVIDIA)

Extract and explain (the teardown):

- **Model:** params · dense or **MoE** (experts, active-per-token) · context length
- **Hardware:** GPU type & count · interconnect (NVLink/NVSwitch, InfiniBand)
- **Framework:** Megatron-LM · DeepSpeed · TorchTitan · custom
- **Parallelism plan:** the **TP** · **PP** · **DP** degrees (+ **EP** / **CP** if used) · which **ZeRO** stage
- **Precision:** BF16 · **FP8** · MXFP4/FP4
- **The clever bits (latest techniques):** communication/computation **overlap** · custom kernels · pipeline schedule (e.g. DualPipe / 1F1B) · activation checkpointing · MoE routing / all-to-all optimization
- **The mapping:** what runs *inside* a node vs *across* nodes — and *why*

Deliverable: a 1-page teardown + the report link. Connect at least **three chapter concepts** to what the real system did.

Part 4 · Explain it (*understanding*) — 15 pts

Short answers (2–4 sentences each):

1. Why do 2 GPUs give ~1.7×, not 2× — and what makes that gap **worse** on the 5090 than on an H100 node?
2. DDP vs FSDP: what does each split, and how do you choose?
3. Why must **tensor parallel** live inside a node while **data parallel** can span the slowest links?
4. What is the **pipeline bubble**, and how do you shrink it?
5. Why is **3D** the name — what are the three coordinates of a GPU?
6. At 1,000 GPUs, why is **checkpointing** non-negotiable, and what does a **recovery** actually do?
7. What does **DCGM** give you that `nvidia-smi` on one box does not?

Part 5 · The Challenge — 3D Parallelism Arena 🏆 (*individual · live leaderboard*) — 10 pts

The competitive finale — **this one is individual**. Each student submits a **parallelism plan** into the **Arena app** (`python arena_app.py`), which **scores it instantly and ranks everyone live**.

Scenario: a **120B dense model** on **128 GPUs** (16 nodes × 8 · 80 GB each · NVLink in-node, InfiniBand across).

You submit: `TP · PP · DP` + precision (BF16/FP8) + activation checkpointing. The app checks:

1. **Valid?** `TP × PP × DP` must equal 128, and **TP must stay inside a node** (≤ 8 , or the NVLink penalty crushes you).
2. **Fits?** memory/GPU ≤ 80 GB (shard more or turn on checkpointing).
3. **Score = MFU** (model-FLOPs utilization): the plan that fits with the **least communication + pipeline bubble** wins.

You get only 3 submissions, and the ranking rewards *understanding, not clicking*: you're ranked by your **best MFU**, and **ties are broken by fewest attempts** — so do the memory math and reason it out *before* you submit. There's a sweet spot: *enough* TP + PP to fit, but not so much that comm and the bubble eat your throughput; the rest goes to **DP**. Write down the plan you settled on and *why* — that reasoning is the graded part; the ranking is glory.

How the score (MFU) is calculated

A plan that is **valid** (`TP × PP × DP = 128` , `TP  $\leq 8$`) and **fits** (≤ 80 GB/GPU) is scored by its **Model-FLOPs Utilization** — the share of compute doing useful work instead of being lost to communication and pipeline idle time. Anything invalid or OOM scores **0**.

$$\text{MFU} = 1 / (1 + \text{tp_overhead} + \text{pipeline_bubble} + \text{dp_overhead}) \quad \rightarrow \text{shown as a \%}$$

Term	Formula	Why
TP overhead	$0.03 \times (TP - 1) \times \text{link_penalty}$	TP all-reduces every layer . <code>link_penalty = 1</code> inside a node (NVLink), but 18× if <code>TP > 8</code> (crosses to InfiniBand)
Pipeline bubble	$(PP - 1) / ((PP - 1) + 16)$	pipeline idle time — grows with PP, shrinks as micro-batches (the <code>16</code>) rise
DP overhead	$0.01 \times \log_2(DP)$	one gradient all-reduce per step → nearly free

Worked example — TP8 · PP4 · DP4: $0.03 \times 7 = 0.21$ + $3/19 = 0.158$ + $0.01 \times \log_2 4 = 0.02$ → MFU = $1 / 1.388 = 72.1\%$.

The memory gate (checked first): `model_state = params × bytes/param` (16 for BF16, 12 for FP8); per-GPU = `model_state ÷ (TP×PP)` + activations $0.8 \times \text{layers}/PP \times (0.35 \text{ if checkpointing})$; the total must be ≤ 80 GB or the plan is invalid.

It's a **simplified teaching model**: it rewards the right decisions — TP inside NVLink, just enough PP to fit, the rest in DP — not exact wall-clock numbers.

Deliverables

- `part1_results.md` — the measurement table + explanations
- `part5_arena.md` — your best plan, its MFU, and why you chose it
- `part2_design.md` — the three parallelism plans (math + mapping)
- `part3_tearardown.pdf` — the 1-page frontier-model teardown + link
- `part4_answers.md` — the short answers

Grading — 100 points

Part	Criterion	Pts
1	Measure — all four labs run, numbers correct, explanations right	20
2	Design — three valid plans with memory math + hardware mapping (EP + CP handled)	20
3	Investigate — accurate teardown, ≥3 chapter concepts connected	25
4	Explain — the seven short answers	15
5	Arena — your best plan + the reasoning behind it (ranking = glory)	10
—	Write-up quality — clear, correct, well-reasoned explanations across all parts	10
—	Bonus — a real multi-node run (RunPod) or a Docker/ <code>kind</code> multi-node simulation, documented	+10

Bonus · see multi-node for real (optional, +10)

Pick one:

- RunPod:** spin up a real **2-node** box and run `train_ddp.py` with `--nnodes=2`; report the cross-node scaling.
- Simulate it:** a `docker compose` mini-Slurm cluster *or* `kind` (Kubernetes-in-Docker) with 2 workers; launch a `torchrun --nnodes=2` job and show ranks 0–15 spanning two "nodes."

Resources you already have

- Multi-GPU labs** — `train_ddp.py` · `train_fsdp.py` · `bench_nccl.py` · `tp_from_scratch.py` · `EXERCISE_plan_the_parallelism.md`
- The Cluster flowchart** — `Cluster_Training_Loop_Flowchart.pdf`
- Deck sections: **Multi-GPU Training** · **Multi-Node & 3D Parallelism**

Appendix · latest topics worth a look (for Part 3)

[FP8 training](#) · [MXFP4 / FP4](#) · [ZeRO-3 / FSDP2](#) · [Expert Parallelism & all-to-all \(DeepEP\)](#) · [Context / sequence parallelism \(ring attention\)](#) · [pipeline schedules \(1F1B, DualPipe, zero-bubble\)](#) · [communication-computation overlap](#) · [TorchTitan / Megatron-LM / DeepSpeed](#) · [4D & 5D parallelism](#)

Measure what you can, design what you can't, and learn from those who did it for real.